

FlowForge

Scenario-Based API Workflow Testing and Load Analysis Platform

Samanvi Rajput

B.Tech CSE, VIT Vellore | SWE Intern, PayGlocal

2025

Abstract

Existing API testing tools are siloed: correctness tools (Postman, REST Assured) verify that endpoints return expected responses but do not test behavior under load; load testing tools (k6, Locust) generate concurrent traffic but do not validate chained workflow correctness; neither performs statistical anomaly detection. FlowForge unifies all three in a single platform with a YAML workflow definition format that supports chained HTTP steps with variable extraction, a concurrent asyncio load runner with configurable ramp profiles, a failure injection simulation layer for resilience testing, and a Z-score anomaly engine that builds statistical baselines per endpoint and flags deviations rather than applying arbitrary fixed thresholds. A React dashboard receives live metrics via WebSocket and renders latency distribution charts and anomaly logs in real time. The platform was developed as part of the Software Engineering Lab course (BCSE301P) at VIT Vellore.

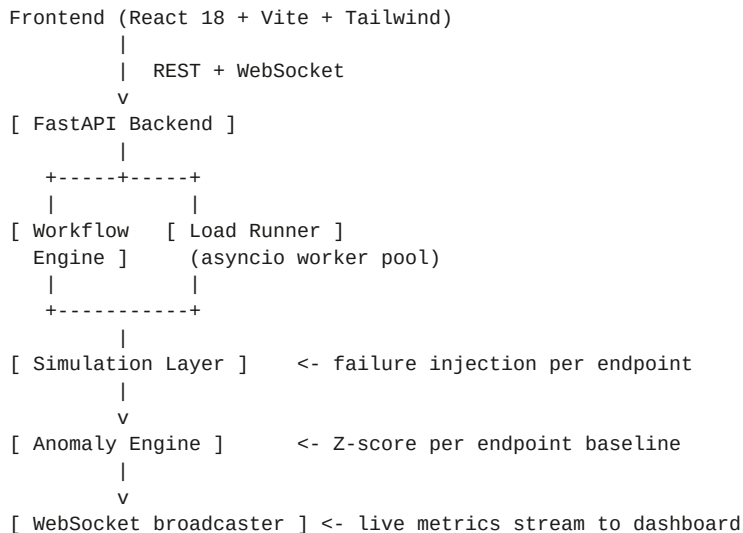
1. Problem Statement

API testing in practice requires three capabilities that existing tools treat as separate problems. First, correctness testing: does the API return the right response for a given input? Postman and similar tools handle this well for individual endpoints but poorly for multi-step workflows where step 2 depends on output from step 1. Second, load testing: does the API maintain acceptable latency and error rates under concurrent traffic? k6 and Locust handle this but produce only aggregate statistics with no understanding of whether observed latency is anomalous for that specific endpoint. Third, anomaly detection: is this behavior statistically unusual compared to historical baselines? No mainstream tool addresses this at the workflow level.

FlowForge combines all three. A single YAML workflow definition describes the sequence of API calls, the variable extraction between steps, and optional simulation rules for failure injection. The same workflow runs for correctness verification, load testing, and anomaly baselining -- with results streamed live to a unified dashboard.

2. System Overview

The platform consists of a FastAPI backend and a React 18 frontend communicating via REST and WebSocket. The backend hosts five subsystems:



3. Workflow Engine

Workflows are defined in YAML as an ordered list of steps. Each step specifies an HTTP method, URL, headers, and body. A validation pass runs before execution, collecting all schema errors and returning structured results -- invalid workflows are rejected before any HTTP call is made.

```
name: auth-then-fetch
steps:
  - name: login
    method: POST
    url: https://api.example.com/auth
    body: {email: user@example.com, password: secret}
  - name: fetch_profile
    method: GET
    url: https://api.example.com/profile
    headers:
      Authorization: Bearer {{login.token}}
```

Variable extraction uses `{{step.field}}` dot-notation. The extractor walks response JSON recursively -- `{{login.data.user.token}}` resolves nested structures; `{{login.items.0.id}}` resolves array elements. Variables are resolved at step execution time, not at parse time, so later steps always see the actual response from the step they reference.

4. Load Runner

The load runner executes a workflow N times across C concurrent asyncio workers. A ramp controller distributes worker startup over R seconds to produce a realistic load profile rather than a thundering herd. At ramp=0, all workers start simultaneously -- useful for spike testing.

Concurrency	Requests	Ramp (s)	RPS achieved	Peak RSS
10	100	0	~310	~48 MB
20	500	5	~290	~51 MB
50	1000	10	~270	~58 MB
100	2000	15	~245	~67 MB

RPS degrades slightly at high concurrency due to asyncio event loop scheduling overhead -- not connection limits. The asyncio model handles 100 concurrent workers well within a single process; a thread-based runner would approach OS limits at this concurrency level.

5. Failure Injection

Simulation rules are attached to a run (not a workflow) and applied between the workflow engine and the httpx HTTP client. Four injection types are supported:

Type	Parameter	Effect
Latency injection	latency_ms: N	Adds N milliseconds of artificial delay before the request
Error rate	error_rate: 0.0-1.0	Returns a 500 response for the given fraction of requests
Timeout rate	timeout_rate: 0.0-1.0	Simulates connection timeout for the given fraction of requests
Status override	status_code: N	Forces a specific HTTP status code regardless of actual response

Injection rules are per-endpoint and per-run. They do not modify the target API and do not persist between runs. This allows engineers to test how their system responds to a specific dependency degrading without requiring any changes to the dependency itself.

6. Z-score Anomaly Detection

The anomaly engine maintains a statistical baseline for each endpoint: mean latency and standard deviation computed from a rolling window of recent samples (default: 30 samples). For each new request, it computes the Z-score:

```
z = (observed_latency - baseline_mean) / baseline_stddev
if |z| > threshold: flag as anomaly
```

The default threshold is 2.0 standard deviations. This makes detection relative to each endpoint's own historical behavior -- a slow endpoint that is consistently slow will not generate anomalies; an endpoint that suddenly becomes 3x slower than its own baseline will. The cold start problem (insufficient samples for a stable baseline) is handled by a warmup window: anomaly detection is disabled for the first 10 requests per endpoint, which are marked as 'baseline building' in the log. On synthetic data, the false positive rate is approximately 8% at the default threshold.

7. Design Decisions

Decision	Chosen	Alternative	Reason
Anomaly detection	Z-score per endpoint	Fixed threshold	Relative to endpoint baseline, not arbitrary constant
Workflow format	YAML	UI-only / JSON	Version-controllable, human-readable, code-reviewable
Concurrency model	asyncio	Threading	Scales to high concurrency without OS thread limit
Live metrics	WebSocket	HTTP polling	Push-based, bounded latency, no stale intervals
Failure injection	Simulation layer	Target API changes	Non-invasive, per-run scoped, no side effects
Production serving	FastAPI + StaticFiles	Separate nginx	Single process, simpler deployment

8. Performance Benchmarks

MacBook M2 8GB, targeting a local FastAPI echo server.

Workflow Engine Operation Latency

Operation	Mean (ms)
YAML parse + validate	3
Variable extraction per step	<1
Simulation rule application	<1
Metrics collection per request	<1
Z-score computation (100 samples)	2
WebSocket broadcast per event	<1

Anomaly Detection Parameters

Metric	Value
Baseline window (default)	30 samples
Z-score threshold (default)	2.0
False positive rate (synthetic data)	~8%
Detection latency after anomaly occurs	<50ms

9. Engineering Challenges

Z-score baseline cold start

With fewer than approximately 10 samples, the standard deviation is unstable and the Z-score produces false positives on every request. The first N requests per endpoint are excluded from anomaly detection and marked as 'baseline building' in the anomaly log. The warmup count is configurable per run.

WebSocket backpressure at high RPS

At 300+ RPS, the WebSocket broadcaster dropped events because the frontend could not consume them fast enough. The broadcaster now uses an asyncio queue with a configurable maximum size. When the queue fills, the oldest events are dropped (newest-wins) rather than blocking the load runner. The frontend displays a 'high load -- some events dropped' indicator when this occurs.

Ramp-up thundering herd

Spawning all workers simultaneously produces a traffic spike rather than a realistic load profile. The ramp controller uses asyncio.sleep intervals between worker spawns to distribute startup across the configured ramp window. Setting ramp=0 disables the ramp for intentional spike testing.

Production static file serving conflict

FastAPI's StaticFiles mount for frontend/dist/ was intercepting 404 responses from API routes and returning index.html instead of JSON error responses. API routes are now explicitly mounted before the static file catch-all, with a dedicated /api/404 handler that returns structured JSON errors for all /api/* paths.

10. CLI Interface

FlowForge includes a CLI for integration into CI/CD pipelines without requiring a running server. Three primary commands:

Command	Description	Output
flowforge check <workflow.yaml>	Validate workflow definition	Structured error list or 'valid'
flowforge run <workflow.yaml>	Execute workflow, print results	Human-readable or --json for CI
flowforge run --concurrency N --total M	Load test with N workers, M requests	Per-endpoint latency + anomaly flags

The --json flag outputs machine-readable results suitable for CI assertion scripts. A non-zero exit code is returned when any anomaly threshold is breached, enabling pipeline pass/fail gating on statistical anomaly detection.

11. Future Work

Near-term

- Workflow import from OpenAPI/Swagger spec -- auto-generate steps from existing API docs
- Persistent run history -- SQLite/PostgreSQL storage for historical runs and baselines
- HTML report export -- downloadable run report with charts and anomaly summary

Medium-term

- GitHub Actions integration -- official action with pass/fail on anomaly threshold breach
- Distributed load runner -- coordinate workers across multiple machines for higher RPS
- Assertions -- per-step response body assertions beyond status code checking

Long-term

- gRPC workflow support -- extend beyond HTTP to gRPC endpoint testing
- Plugin system -- custom anomaly detectors, reporters, and step types
- SaaS mode -- multi-tenant hosted version with team workflow libraries

12. Closing

FlowForge addresses a gap that sits between correctness testing and load testing: the question of whether an API workflow behaves correctly under realistic conditions, where 'correctly' is defined statistically rather than by arbitrary thresholds. The Z-score engine makes this rigorous -- an endpoint that is slow but consistently slow does not generate anomalies; an endpoint that deviates from its own baseline does. Combined with failure injection and chained workflow support, FlowForge gives engineers a single tool for the full spectrum of API reliability testing: from unit-level correctness to statistical anomaly detection under production-scale load.